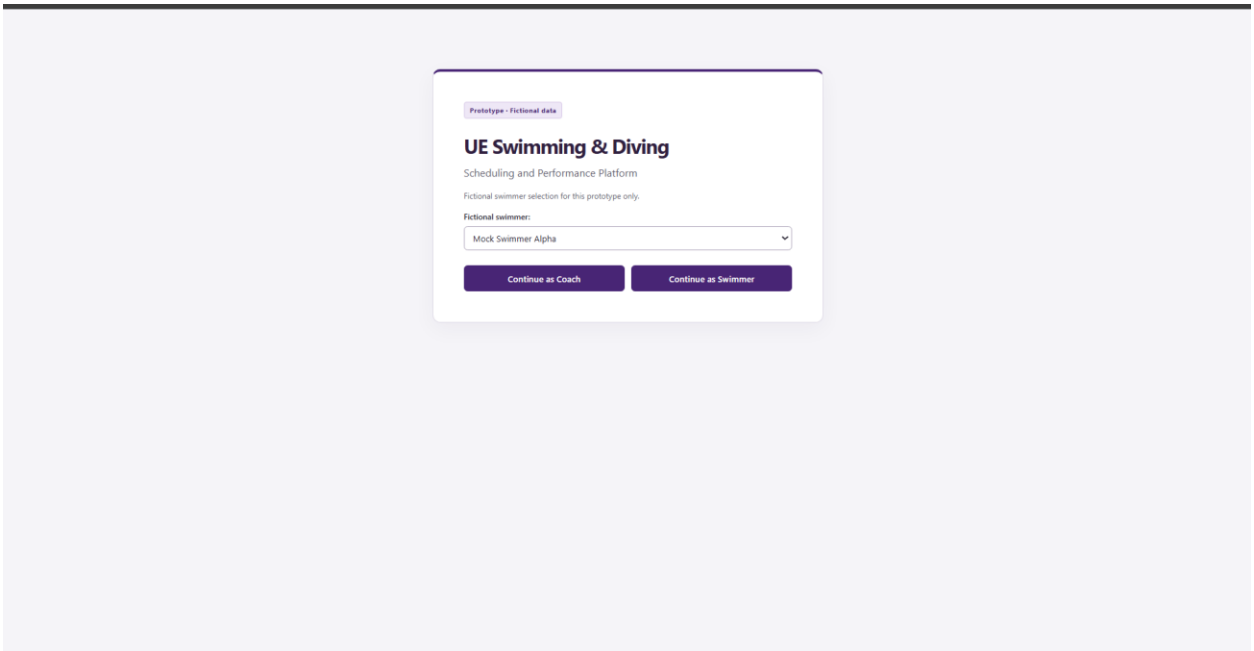
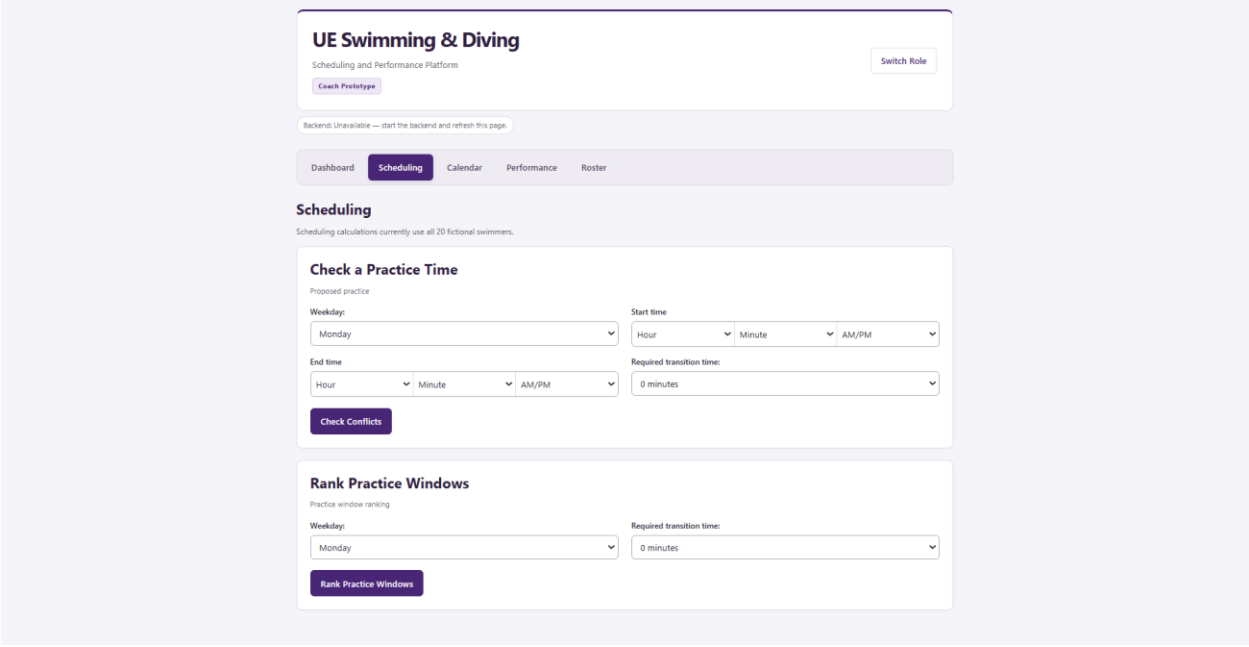
Demo Video: https://youtu.be/yuZ8MBSPVbA

Screenshots:

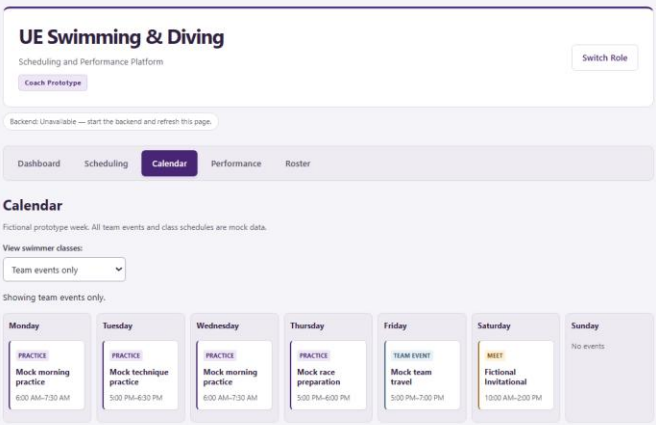
1. **Landing Screen:**



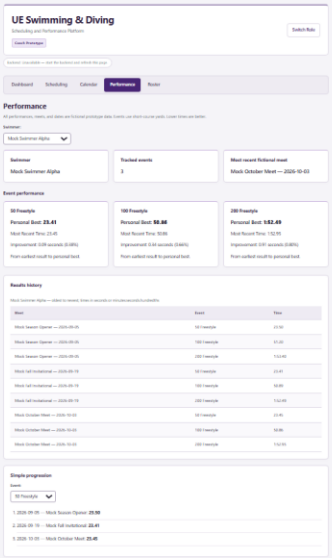
2. **Coach Scheduling:**



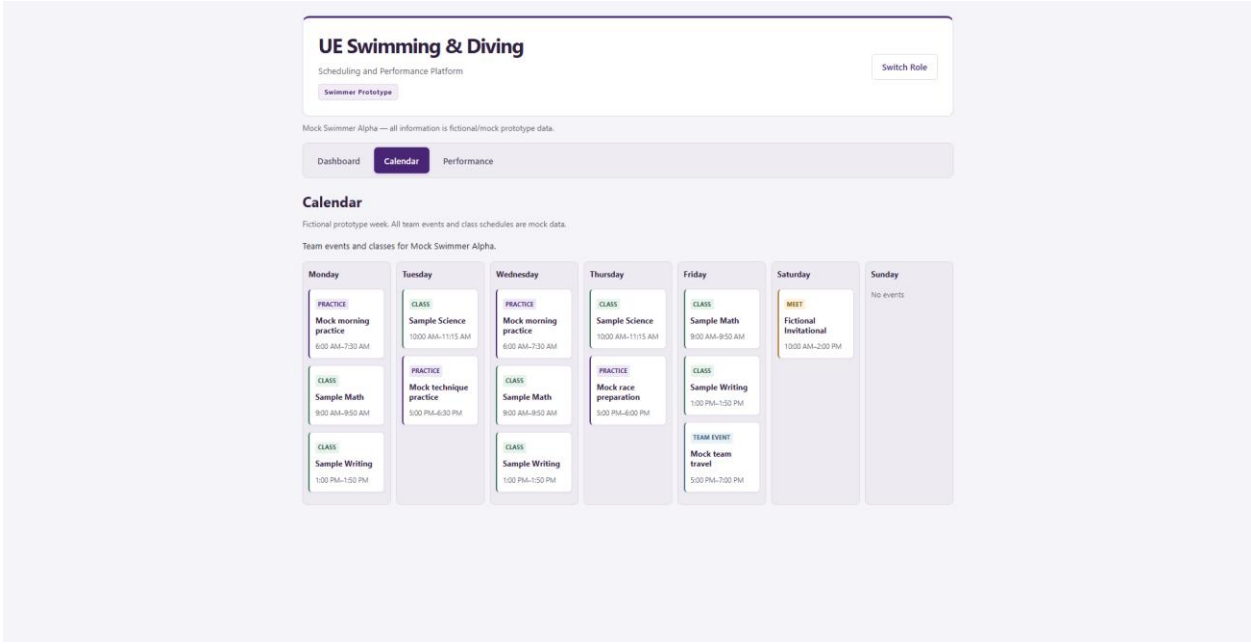
3. **Coach Calendar:**



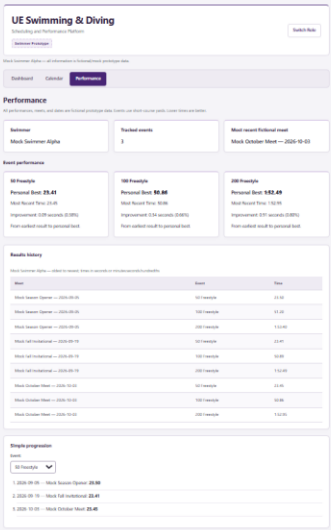
4. **Coach Performance:**



5. **Swimmer Calendar:**



6. **Swimmer Performance:**



GitHub:

https://github.com/michael-pruett-9/ue-swim-platform

Project Title:

Swim Team Scheduling and Performance Platform for University of Evansville Swimming & Diving

Project Summary:

My senior project is a scheduling and performance platform for the University of Evansville Swimming & Diving team. The goal is to combine scheduling and swimming performance information that is currently spread across

multiple systems. Prototype v1 currently focuses on practice scheduling, conflict detection and ranking, team calendars, and basic swimmer performance tracking using fictional data.

Connection To Requirements Document:

The current prototype was built based on the user, functional, and non-functional requirements that were identified earlier within the project during the requirements documentation. The highest priority requirements for the project that I set was to allow coaches to view swimmer schedules, identify scheduling conflicts, compare possible practice times, and eventually view swimming performance information in the same system. Prototype v1 is a baseline for all those major requirements while using fictional data. Coaches can inspect swimmer schedules, check whether a proposed practice conflicts with athletes' classes, rank predefined practice windows according to the number of conflicts, browse a weekly team calendar, view individual swimmers, and inspect fictional swimming performances. The prototype also includes a separate view made for swimmers which allows them to view their own schedule and performance. I wanted this system to be something not only for the coaches, but for the entire team and this is a huge step towards that goal. I wanted to ensure though that coaches and swimmers have different levels of access as a swimmer should not be able to do as much as a coach can. The protoype demonstrates these separate Coach and Swimmer views.

Functional Requirements and Current Status:

Several of the most important functional requirements have already been implemented within Prototype v1. The system contains 20 fictional swimmers with fictional class and practice schedules during a school week. Coaches can enter a proposed practice day and time and the system can identify which swimmers have conflicts with that proposed time. The system supports Monday through Sunday and includes an optional transition time that can fit coaches' constraints to allow swimmers a certain amount of time between practice and other events. The platform can also rank a set of possible practice windows by the number of conflicts it has per day. This provides the foundation for the eventual goal of allowing coaches to identify the best possible times for practice for the team overall. The prototype has two modes with the coach and swimmer modes. The coach mode currently contains Dashboard, Scheduling, Calendar, Performance, and Roster sections. The weekly calendar displays fictional practices, travel, and a meet. A coach can also select a swimmer and overlay that swimmer's class schedule onto the team calendar. The performance section contains fictional data which simulates a college swimming team's team dynamic and progression. The application calculates personal bests, recent results, improvement, historical results, and basic progression information. The swimmer mode contains a Dashboard, Calendar, and Performance section. These work the exact same way as the coach's mode but only allow the swimmer to view their own personal schedule and performance. A fictional swimmer is selected before entering since a login/authentication method has not been implemented yet. Some planned functionality has not yet been implemented. Athlete and team goal tracking, optional practice tracking, database persistence, real authentication, and external data integration remain future work.

Non-Functional Requirements and Current Status:

One important consideration during the project's development has been usability. The original proof of concept was a very simple visual of what the project would entail, but Prototype v1 contains a more consistent interface with cards, navigation, forms, labels, tables, and responsive styling. Scheduling information is displayed with text rather than just color which is important for accessibility and makes conflicts as well as results easier to interpret. The current system operates locally and responds quickly with the fictional dataset for the prototype. The backend scheduling logic is tested through Python unit tests. The frontend can be checked by using its production build command to verify that the React application builds successfully. Privacy and security requirements are not yet fully implemented because Prototype v1 does not contain authentication or a production database. Instead, the prototype uses fictional data so that no real student-athlete information is exposed during development.

What I Have Accomplished:

The biggest accomplishment during this phase was moving from the basic proof of concept to the current prototype which much better resembles the larger system I originally proposed. The early proof of concept had only 5 fictional swimmers with schedules from Monday to Tuesday. This dataset was displayed in a very simple way and was used to check the conflict detection logic. Since then, the system now has 20 fictional swimmers, Monday-Sunday, transition-aware conflict detection, practice-window ranking, separate Coach and Swimmer experiences, a roster, a weekly calendar, and performance tracking. I also have improved the interface significantly, making it much cleaner than before and making it behave much more like an actual application rather than just a basic technical demonstration. The frontend uses React/Vite to handle the interface while Python/FastAPI handles the scheduling calculations. Automated backend tests are also being used to make sure changes do not break existing scheduling functionality.

What I Have Learned:

One of the main things I have learned from this stage of development was just how important incremental project development really is. If I had tried to build the entire project at once rather than moving in smaller incremental steps it would have made the project much harder to understand and test. I started with the basic conflict detection and display then gradually expanded the project while preserving the parts that already worked. I have also learned a lot more about connecting frontend and backend through API requests. Working on scheduling logic required me to think carefully about time intervals, overlapping schedules, transition times, and edge cases. Another important lesson for me was that functionality and usability need to develop together. A feature can work but still be inconvenient to use, which is very interesting. One example was the original browser time input. I later replaced it with clearer hour, minute, and AM/PM selectors to make scheduling easier for a coach to use.

Challenges:

One of the biggest challenges I have had during my project's development was controlling the scope of the project. I want the complete project to include scheduling, performance tracking, calendars, goals, potential practice tracking, permissions, and possible external integrations. Trying to implement all of those features at the same time makes it easy to lose focus and forget the features that are the most important. Another challenge was that the scheduling system required careful handling of edge cases. Classes can partially overlap practices, completely contain them, or be immediately before or after them. Adding configurable transition time made these calculations more complicated. Another challenge was designing the project around the future real data I will use while using fictional data. The current application needs to demonstrate the intended workflow without assuming that access to systems such as SwimCloud or other university systems will be available.

Success:

The core scheduling features have been implemented successfully. Conflict detection, ranking practice windows, and transition times based on coach's preference have all been implemented into Prototype v1. The project has expanded substantially without losing the original scheduling functionality. Coach and Swimmer prototype views now clearly show how different users could interact with the system. Another success is that the prototype represents both major sides of what I wanted when I originally thought up the idea for the project. It has scheduling and swimming performances. These may not be perfect or exactly what I want yet, but Prototype v1 demonstrates that they can coexist within one platform.

Responsible Computing Concerns:

The concerns for the project have not changed as student-athlete schedules and performance could contain sensitive private information that I must ensure that the project protects accordingly. Prototype v1 uses fictional data including fictional swimmers, schedules, team events, and performance data. No real student-athlete information is

required to demonstrate the current system. In a future version the project will need to have Coach and Swimmer accounts with real authentication and role-based permissions. The system should also avoid presenting performance information in a way that could unfairly compare or negatively affect swimmers without context. Scheduling recommendations should also remain recommendations rather than automatic decisions. Coaches should be able to see why a time was recommended and which athletes would be affected before making a final decision. The system is being built to help not take over.

Next Steps:

-Immediate:

- Generate and rank practice windows automatically using coach constraints
- Replace simple progression lists with performance graphs
- Begin relational database implementation

-Later:

- Goals and practice tracking
- Authentication / real permissions
- Approved data imports
- Deployment and usability testing

AI Use Statement:

AI was used during development for assistance, brainstorming, and troubleshooting. AI assistance did not replace the project requirements, client decisions, testing, or my responsibility for understanding and evaluating the system.